TRIBHUVAN UNIVERSITY

Institute of Engineering, Purwanchal Campus

Department of Electronics and Computer Engineering

Dharan, Sunsari

ARTIFICIAL INTELLIGENCE [ENCT 305]

LAB REPORT: TO IMPLEMENT LOGISTIC REGRESSION ON A GIVEN DATASET AND UNDERSTAND ITS WORKING.

Submitted By:

Abhishek Niraula

Roll no.: PUR080BEI004

Faculty: BEI

Submitted To:

Er. Sujan Karki

Instructor

Purwanchal Campus, Dharan

INTRODUCTION:

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression which predicts continuous values it predicts the probability that an input belongs to a specific class.

It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1.

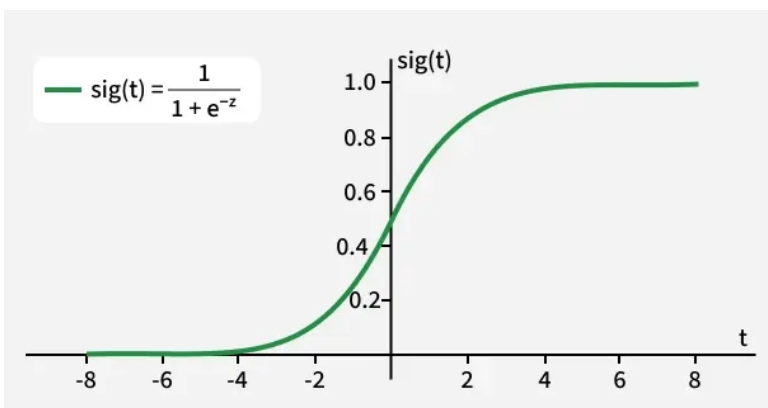
It uses **sigmoid function** to convert inputs into a probability value between 0 and 1.

Assumptions of Logistic Regression

Understanding the assumptions behind logistic regression is important to ensure the model is applied correctly, main assumptions are:

- **Independent observations**: Each data point is assumed to be independent of the others means there should be no correlation or dependence between the input samples.
- **Binary dependent variables**: It takes the assumption that the dependent variable must be binary, meaning it can take only two values. For more than two categories **SoftMax** functions are used.
- **Linearity relationship between independent variables and log odds**: The model assumes a linear relationship between the independent variables and the log odds of the dependent variable which means the predictors affect the log odds in a linear way.
- **No outliers**: The dataset should not contain extreme outliers as they can distort the estimation of the logistic regression coefficients.
- **Large sample size**: It requires a sufficiently large sample size to produce reliable and stable results.

Sigmoid Function



The **sigmoid function** converts the continuous variable data into the probability i.e between 0 and 1.

$$\sigma(z) \text{ tends towards 1 as } z \to \infty$$

$$\sigma(z) \text{ tends towards 0 as } z \to -\infty$$

$$\sigma(z) \text{ is always bounded between 0 and 1}$$

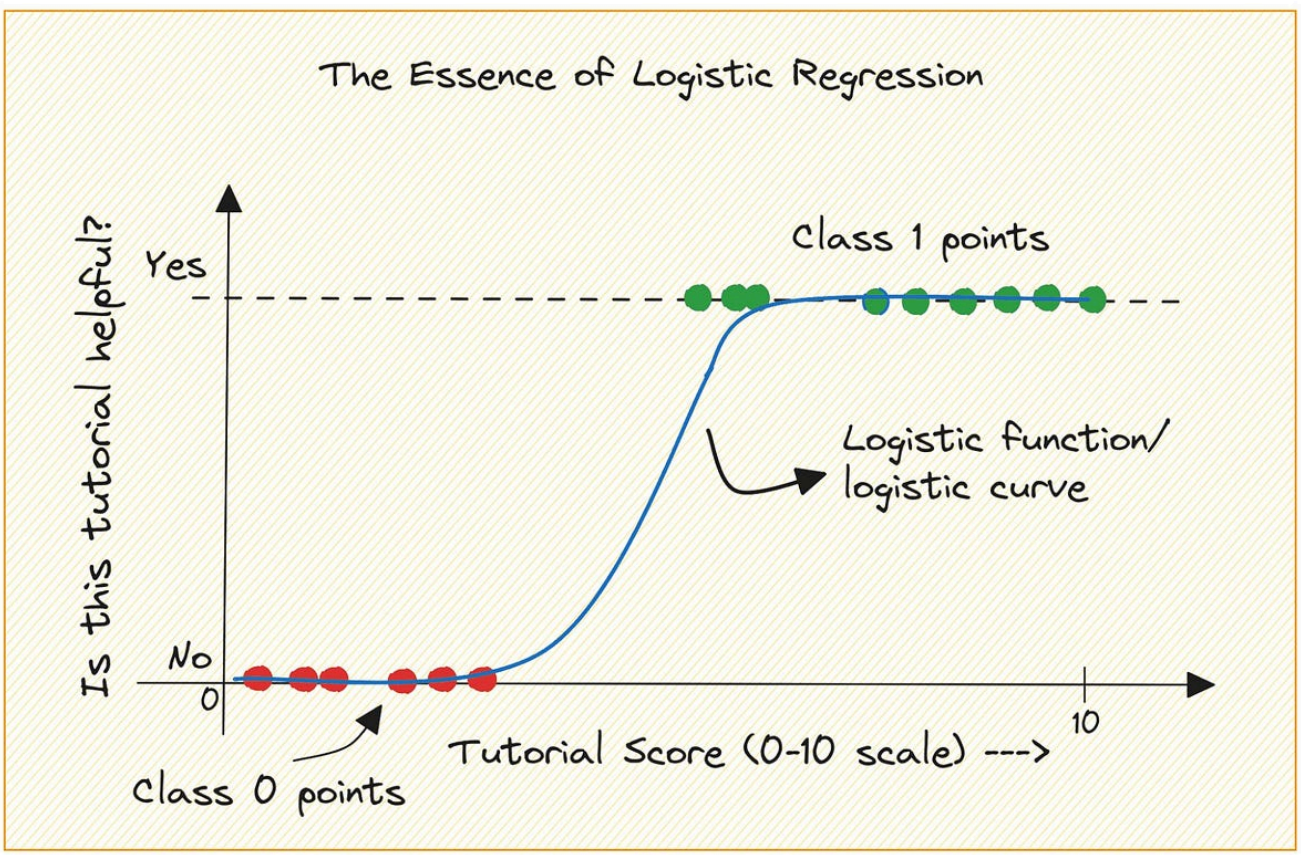
where the probability of being a class can be measured as:

$$P(y=1) = \sigma(z)$$

$$P(y=0) = 1-\sigma(z)$$

In logistic regression, we use a threshold value usually 0.5 to decide the class label.

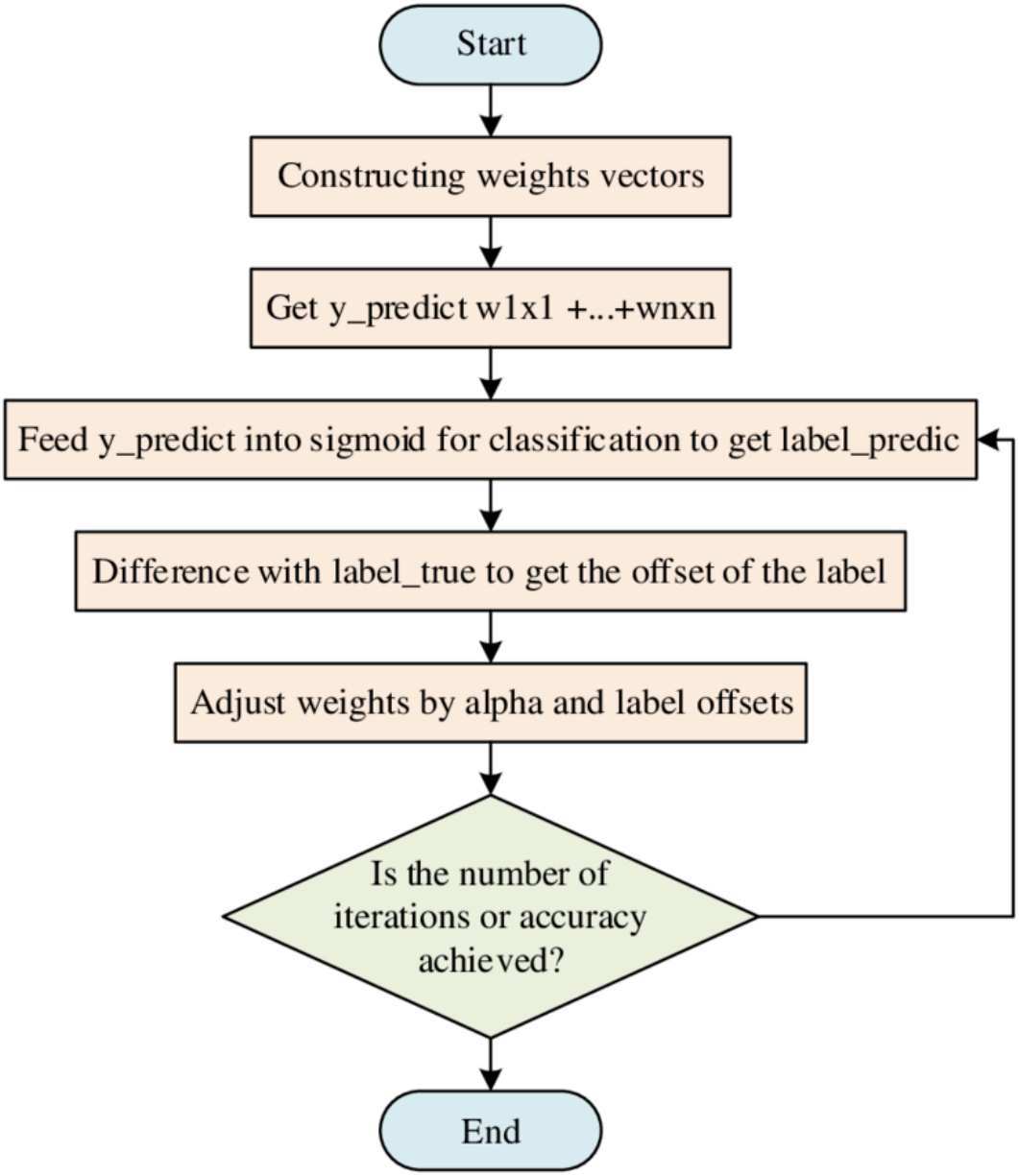
- If the sigmoid output is the same or above the threshold, the input is classified as Class 1.
- If it is below the threshold, the input is classified as Class 0



OBJECTIVES

A) To understand and implement the concept of Logistic Regression in Artificial Intelligence.
B) To train a Logistic Regression model using Python Programming Language.
C) To learn about Sigmoid Function, Accuracy, Confusion Matrix in a Supervised Classification Algorithm.
D) To visualize the model performance graphically.

ALGORITHM



SOURCE CODE

Overview

For the implementation of the Logistic Regression, the dataset was taken from Kaggle which was uploaded by **Ananya Nayan.** This dataset contains information about individuals and their response to a particular advertisement campaign on social media. The dataset includes the following columns:

- Age: Age of the individual in years. (Wasn't used for training model)
- EstimatedSalary: Estimated salary of the individual.
- Purchased: Binary variable indicating whether the individual made a purchase (1) or not (0) after seeing the advertisement.

CODE

main.py

```
# Importing necessary libraries
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import StandardScaler
from model import LogisticRegression

# Reading the data
st.title("Logistic Regression")

data = pd.read_csv("Social_Network_Ads.csv")

# Removing the initial data titles.
data = data.iloc[:, 1: ]

# Get the actual values.
x = data[["Age", "EstimatedSalary"]].values
y = data["Purchased"].values

# Scaling the features to have mean=0 and standard deviation=1.
# This is necessary because features have different scales (AGE: 20-80, IQ:
10000-150000).
# Scaling ensures faster convergence in gradient descent and prevents
```

```
features with larger values from dominating.
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

col1, col2 = st.columns(2)

with col1:
    st.subheader("Dataset Preview")
    st.dataframe(data)

with col2:
    st.subheader("Data Visualization")
    fig, ax = plt.subplots()

    scatter = ax.scatter(x[:, 0], x[:, 1], c = y)
    ax.set_xlabel("Age")
    ax.set_ylabel("EstimatedSalary")

    st.pyplot(fig)

st.sidebar.header("Model Hyperparameter")
lr = st.sidebar.slider("Learning Rate", 0.001, 1.0, 0.1)

n_iters = st.sidebar.slider("Number of Iterations", 1000, 5000, 2000)

# If button pressed, then begin the model fitting.
if st.button("Train Model"):
    x_train, x_test, y_train, y_test = train_test_split(x_scaled, y,
test_size=0.3, random_state=42)
    model = LogisticRegression(lr, n_iters)
    model.fit(x_train, y_train)

    y_pred = model.prob(x_test)

    accuracy = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    st.session_state["model"] = model
    st.session_state["scaler"] = scaler
    st.session_state["accuracy"] = accuracy
    st.session_state["cm"] = cm
    # st.session_state["loss"] = model.loss_history
    st.session_state["trained"] = True
```

```
if "trained" in st.session_state:
    st.subheader("Model Performance")

    col1, col2 = st.columns(2)

    with col1:
        st.metric("Accuracy", f"{st.session_state['accuracy']:.4f}")

    with col2:
        st.write("**Confusion Matrix:**")
        st.dataframe(pd.DataFrame(st.session_state['cm'],
                                  columns=['Predicted 0', 'Predicted 1'],
                                  index=['Actual 0', 'Actual 1']))

    st.subheader("Training Loss Curve")
    fig, ax = plt.subplots()
    ax.plot(st.session_state["model"].loss_history)
    ax.set_xlabel("Iterations")
    ax.set_ylabel("Loss")
    ax.set_title("LOSS VS ITERATIONS")
    st.pyplot(fig)

st.subheader("Predict")
age_input = st.number_input("Enter Age", 18, 65, 30)
salary_input = st.number_input("Enter Estimated Salary", 15000, 150000,
50000)

if st.button("Predict Purchase"):
    if "model" not in st.session_state:
        st.error("You have not trained your model")
    else:
        model = st.session_state["model"]
        scaler = st.session_state["scaler"]
        value = np.array([[age_input, salary_input]])
        value_scaled = scaler.transform(value)

        probability = model.predict(value_scaled[0])
        prob_value = probability.item() if isinstance(probability,
np.ndarray) else probability

        pred = model.prob(value_scaled[0])
        pred_value = pred.item() if isinstance(pred, np.ndarray) else pred
```

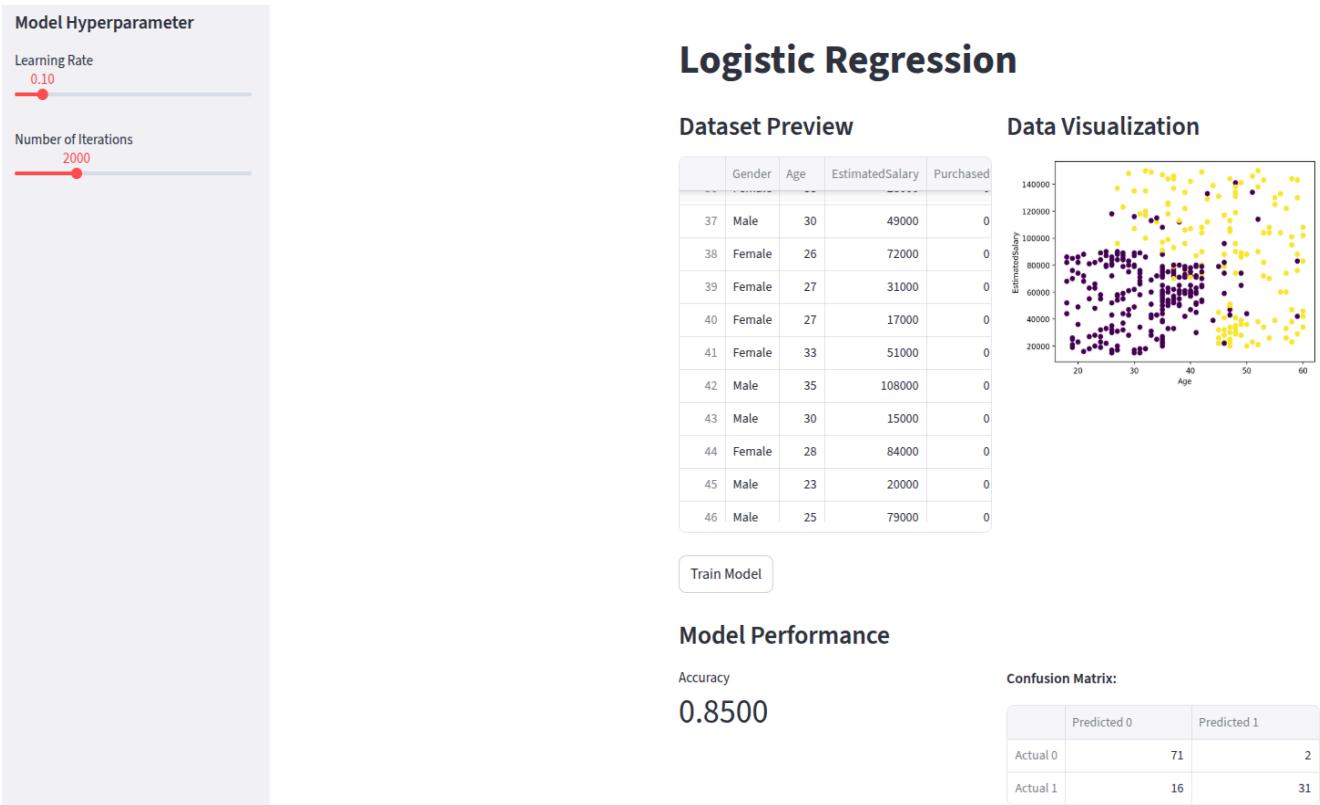
```
        # Display probability
        st.info(f"**Purchase Probability: {prob_value*100:.2f}%**")

        if pred_value == 1:
            st.success("User will purchase")
        else:
            st.warning("User will not purchase")
```

model.py

```
import numpy as np

def sigmoid(x):
    return 1/(1 + np.exp(-x))

class LogisticRegression:
    def __init__(self, lr = 0.01, n_iters = 1000):
        self.lr = lr
        self.n_iters = n_iters
        self.weights = None
        self.bias = None
        self.loss_history = []

    def fit(self, x, y):
        n_samples, n_features = x.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        for _ in range(self.n_iters):
            y_pred = np.dot(x, self.weights) + self.bias
            prediction = sigmoid(y_pred)

            dw = (1/n_samples) * np.dot(x.T, (prediction - y))
            db = (1/n_samples) * np.sum(prediction - y)

            self.weights -= self.lr * dw
            self.bias -= self.lr * db

            # Loss
            loss = - np.mean(y * np.log(prediction + 1e-15) + (1 -
y)*np.log(1 - prediction + 1e-15))
            self.loss_history.append(loss)
```

```python
    def predict(self, x):
        pred = np.dot(x, self.weights) + self.bias
        predicted = sigmoid(pred)
        return predicted

    def prob(self, x):
        probability = self.predict(x)
        return np.where(probability >= 0.5, 1, 0)
```

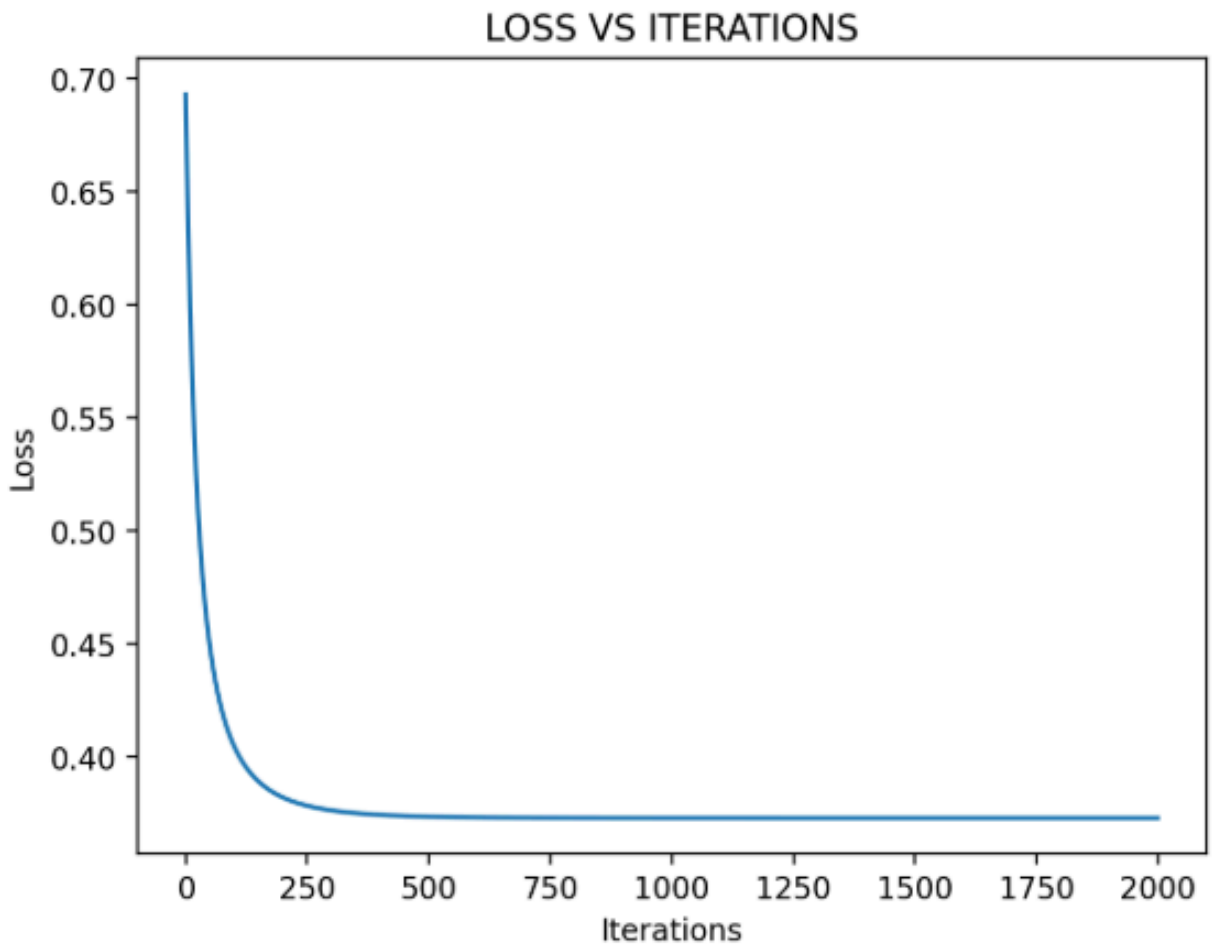
OUTPUT



	Gender	Age	EstimatedSalary	Purchased
37	Male	30	49000	0
38	Female	26	72000	0
39	Female	27	31000	0
40	Female	27	17000	0
41	Female	33	51000	0
42	Male	35	108000	0
43	Male	30	15000	0
44	Female	28	84000	0
45	Male	23	20000	0
46	Male	25	79000	0

	Predicted 0	Predicted 1
Actual 0	71	2
Actual 1	16	31

Training Loss Curve



Predict

Enter Age

30 – +

Enter Estimated Salary

50000 – +

Predict Purchase

Purchase Probability: 3.48%

User will not purchase

CONCLUSION

In this lab, we successfully implemented a supervised classification algorithm: **Logistic Regression**. Through this implementation, we observed how logistic regression models the probability of a binary outcome by applying the **sigmoid function** to a linear combination of input features. This allowed us to understand how classification decisions are made based on probability thresholds rather than direct numerical predictions.

We also explored components such as **feature scaling**, **gradient descent optimization**, **loss computation**, and **decision boundaries**. We also evaluated the model using key performance metrics like **accuracy** and the **confusion matrix**, which helped us interpret how well the classifier distinguishes between the two classes. Additionally, visualizing the **loss curve** provided insight into the convergence behavior of the model during training.